National University of Computer & Emerging Sciences

Course: CS – 4015 Agentic AI (Section: A, B, C)

Homework – II

RAG in the Wild – A case study

Deadline: 15th March 2026 - Sunday

The Scenario

- **The product:** A smart assistant that answers factual questions by searching a large, live snapshot of the web - finance, music, movies, sports, and general knowledge. Users ask everything from "Who directed **Inception*?" to "Which athlete has won more Grand Slams, Federer or Nadal?" and expect accurate, grounded answers.
- **The problem:** The backend doesn't search the live web at query time. Instead, it relies on a ***pre-crawled corpus***: for thousands of past queries, someone ran a search engine and stored the top result pages (snippets and full HTML). So, the "knowledge" is real web content - but ***relevance is not guaranteed***. Some snippets are spot-on; others are ads, related-but-wrong articles, or fragments that only ***look*** relevant. Worse, questions are diverse: simple lookups, comparisons, multi-hop reasoning, and questions with conditions. A naive "grab the top few chunks and generate" pipeline often retrieves the wrong stuff and then confidently hallucinates. The team has seen it happen.
- **Your role:** You've been asked to ***design and compare several advanced RAG strategies*** on this exact corpus. The goal is not to "implement the dataset," but to find out ***which retrieval strategy holds up best when the index is noisy and the questions are varied*** - and to back that up with numbers and a small tool so stakeholders can try queries and see what each pipeline retrieves and answers.

You will build a ***single global corpus and embedding index*** from this snapshot, then implement ***four strategies*** that all use the same index in different ways:

- ✓ **RAG Fusion**
- ✓ **HyDE**
- ✓ **CRAG (Corrective RAG)**
- ✓ **Graph RAG**

You will evaluate each on a held-out dev set and provide a minimal web frontend so people can run queries and inspect retrieved chunks and generated answers. The deliverable is a clear, evidence-based story**: *which technique works best under these conditions, and why.***

Important: Folder Structure (Fixed)

You must not change the folder structure. Do not rename, add, or remove the following directories or files. Only implement the logic inside the indicated modules.

```
├── dataset/
│   └── crag_task_1_and_2_dev_v4.jsonl
```

```
├── docs/
│   └── dataset.md
├── src/
│   ├── data_loader.py          # Load dataset; yield (query, answer, alt_ans, search_results) per example
│   ├── corpus.py               # Build global corpus from dataset; embed all chunks; build/save/load index
│   ├── retrieval.py            # Retrieve top-k chunks from global index given query (embed query, then search index)
│   ├── pipelines/
│   │   ├── __init__.py
│   │   ├── crag.py             # Corrective RAG: confidence-based retrieval decision
│   │   ├── graph_rag.py        # Graph RAG: graph-augmented retrieval over corpus
│   │   ├── rag_fusion.py       # RAG Fusion (e.g. RRF)
│   │   └── hyde.py             # HyDE
│   ├── generation.py           # LLM answer generation from (query, retrieved context)
│   └── evaluation.py           # Compare prediction to answer/alt_ans; accuracy
├── frontend/
│   ├── package.json
│   ├── vite.config.js
│   ├── index.html
│   └── src/
│       ├── main.jsx
│       └── App.jsx             # Query input, pipeline selector, results display
├── config/
│   └── config.example.yaml
├── run_evaluation.py           # Build/load index; run all pipelines on dev set; report accuracy
├── requirements.txt
├── ASSIGNMENT.md
└── README.md                   # Setup and how to run
```

Your Task (Two Phases)

Phase 1: Build the global corpus and index (offline, once)

1. **Load the data** using `data_loader`: iterate all rows.

2. **Extract passages:** From every row, collect all retrieval units (e.g. each `page_snippet` from `search_results`, or chunk `page_result` if you use full HTML). This forms the **global corpus** of chunks.

3. **Embed all chunks** with your chosen embedding model (e.g. sentence-transformers).

4. **Build an index** (e.g. in-memory list of (chunk_text, embedding) with cosine search, or FAISS/numpy). Optionally **save the index** to disk so you can load it without recomputing embeddings.

5. **Expose:** `build_index(dataset_path, ...)` and `load_index(index_path, ...)`; index has `retrieve(query_embedding, top_k) -> list of (chunk_text, score)`.

Implement this in `**src/corpus.py**`.

Phase 2: For each query — retrieve and generate (online)

For each user question (from the dev set or the frontend):

1. **Retrieval:** Using one of the four strategies below, obtain **top-k chunks** from the **global index** (not from a per-question subset).

2. **Generation:** Pass the query and the retrieved chunks to an LLM; return the generated answer.

3. **Evaluation (dev set only):** Compare the answer to gold `answer` and `alt_ans`; compute accuracy per pipeline.

Implement retrieval in `**src/retrieval.py**` (query → embed → index.retrieve). Implement each strategy in `**src/pipelines/*.py**`.

Data at a Glance

- **Path:** `dataset/crag_task_1_and_2_dev_v4.jsonl` (you download and place it; see README).

- **Format:** One JSON object per line. See `docs/dataset.md` for the full schema.

- **Corpus:** Each row has `search_results` (list of items). Each item has `page_snippet` (short text) and `page_result` (full HTML). Use **page_snippet** as one chunk per result, or extract text from `page_result` and chunk if needed. **All chunks from all rows** together form the global corpus.

- **Evaluation:** Each row has `query`, `answer`, and `alt_ans`. You run each pipeline on each query, get a predicted answer, and compare to `answer` and `alt_ans`.

The Four Pipelines (all retrieve from the global index)

1. **RAG Fusion**

Query → generate multiple query variants → **retrieve ranked lists from the global index for each** → **merge the ranked lists** using a fusion method (e.g. **Reciprocal Rank Fusion (RRF)**) → take top-k from the fused list → LLM generates answer from fused context.

2. **HyDE (Hypothetical Document Embedding)**

Query → LLM generates a **hypothetical document** (1–2 paragraphs) that might contain the answer → embed the **hypothetical doc** (not the query) → **retrieve top-k from the global index** by similarity to this embedding → LLM generates the final answer from retrieved context.

3. **CRAG (Corrective RAG)**

Query → embed query and **retrieve** from the global index as usual. Then **assess retrieval confidence** (e.g. with an NLI model, consistency check, or LLM judge): if confidence is **high**, use the retrieved chunks for generation; if **low**, either skip retrieval and generate from the query alone, or fall back to a different strategy (e.g. fewer chunks or web-style expansion). The idea is to ***correct*** the pipeline based on whether retrieval is likely helpful.

When producing the final answer, students must include source of information used from the retrieved context and cite it at the end of the answer using any standard citation style (e.g. IEEE, APA etc)

4. **Graph RAG**

Build or use a **graph view** of the corpus (e.g. entities and relations extracted from chunks, or document/chunk similarity graph). For a query, **retrieve in a graph-aware way** (e.g. entity linking, subgraph retrieval, or spreading from seed nodes), then turn the selected graph neighborhood into text and pass it to the LLM for answer generation. All retrieval still uses the same underlying corpus; the difference is that retrieval is **graph-augmented** rather than pure vector search.

Evaluation

- Implement `src/evaluation.py`: compare predicted answer to `answer` and `alt_ans` (exact or normalized match).

- Implement `run_evaluation.py`: build or load the global index once; for each dev example, run all four pipelines (each retrieves from the global index); compute **accuracy per pipeline**; print or save results.

- Display the **retrieval score (similarity score or confidence score)** for the retrieved chunks along with the generated answer. This score should be visible either in the evaluation output or in the frontend interface when results are displayed.

Frontend

- The frontend is a **React** app (Vite). Implement the UI in `frontend/src/App.jsx` so that a user can:

  - Enter a **query** (or choose a sample from the dataset).

  - Select one of the **four pipelines** (RAG Fusion, HyDE, CRAG, Graph RAG).

  - Run and see **retrieved chunks** (from the global index) and the **generated answer**.

- You will need a backend (e.g. Flask or FastAPI) that loads the index and runs the selected pipeline; the React app calls this backend.

Deliverables

1. **Code:** All implementations in the prescribed files and folders (no structure changes).

2. **Evaluation results:** Output or file from `run_evaluation.py` showing accuracy for each pipeline.

3. **Short report (e.g. 1–2 pages):** Written as a **recommendation to the team**: describe each pipeline in one paragraph, then say which strategy you would ship for this product and why - backed by your accuracy numbers and any patterns you noticed (e.g. fusion helping on multi-hop questions, confidence gating reducing hallucinations when retrieval is off).

References

- Dataset schema: `docs/dataset.md`

- CRAG (Corrective RAG) and Graph RAG: see course material or standard references for confidence-based retrieval and graph-augmented RAG.